

3교시: 미니 앱 구현 1 - HTML 구조, CSS, JS, dummy JSON 연결

수업 목표

- skeleton을 실제 화면 구조로 확장한다.
- dummy JSON을 `fetch` 로 읽어 화면에 렌더링한다.
- UI, data, script의 책임을 분리한다.

50분 운영

시간	활동	학습 초점	학생 산출
0-5분	skeleton 실행 확인	서버가 켜진 상태를 확인한다.	실행 URL
5-15분	HTML 영역 설계	title, controls, list, status 영역을 잡는다.	semantic section
15-25분	CSS 기본 정리	읽기 쉬운 layout과 상태 색을 만든다.	기본 스타일
25-40분	JS fetch/render	JSON을 읽고 DOM에 표시한다.	렌더링 코드
40-50분	증거 캡처	browser 화면과 console 오류를 확인한다.	구현 evidence

0-5분 skeleton 실행 확인

- 초점: 서버가 켜진 상태를 확인한다.
- 학생 산출: 실행 URL

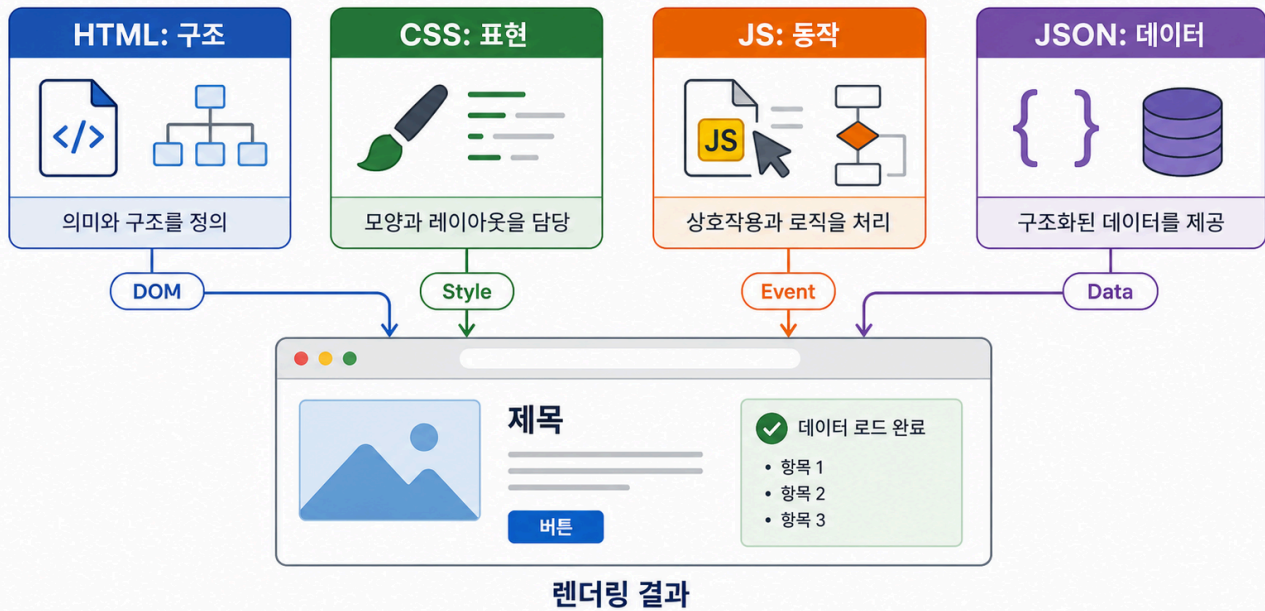
핵심 설명

이 교시의 완성 기준은 예쁜 디자인이 아니라 데이터가 파일에서 화면으로 이동하는 흐름을 증명하는 것이다. 학생은 `data.json -> fetch -> DOM rendering -> browser 확인` 흐름을 말로 설명할 수 있어야 한다.

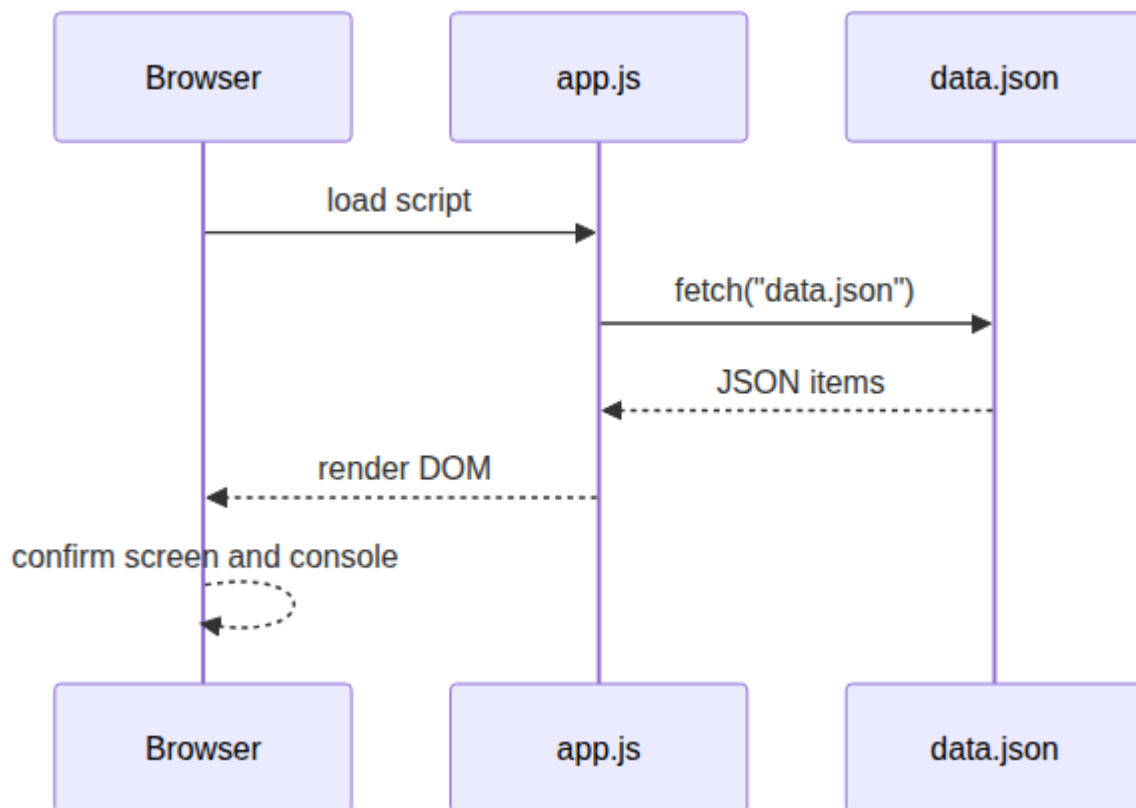
Visual 1: 구조 다이어그램

HTML · CSS · JS · JSON 책임 분리

각 기술은 한 가지 책임만 맡고, 브라우저가 통합해 화면을 렌더링합니다.



이 이미지는 브라우저 화면이 한 파일에서 생기는 결과가 아니라 구조, 표현, 동작, 데이터가 합쳐진 렌더링 결과임을 보여준다. 이후 디버깅할 때 어느 파일을 먼저 볼지 판단하는 기준으로 사용한다.



5-15분 HTML 영역 설계

- 초점: title, controls, list, status 영역을 잡는다.

- 학생 산출: semantic section

Visual 2: 렌더링 관찰 지점

관찰 지점	확인 질문	evidence 예시
HTML	렌더링 대상 영역이 있는가?	#app 영역 캡처
JS	fetch("data.json") 경로가 맞는가?	코드 스니펫 또는 파일 경로
Data	JSON 배열이 유효한가?	item 3개 표시
Browser	화면과 console이 정상인가?	화면 캡처와 console 메모

15-25분 CSS 기본 정리

- 초점: 읽기 쉬운 layout과 상태 색을 만든다.
- 학생 산출: 기본 스타일

Visual 3: 책임 분리 카드

파일	담당 책임	섞이면 생기는 문제
HTML	구조	데이터와 스타일 변경이 어려움
CSS	표현	상태 변화가 코드에 숨음
JS	동작과 렌더링	경로/데이터 오류 추적 어려움

활동 절차

1. index.html 에 앱 제목, 설명, 목록 영역, 상태 메시지 영역을 만든다.
2. styles.css 에 body, main, list, status class를 정의한다.
3. app.js 에서 fetch("data.json") 로 데이터를 읽는다.
4. 각 item을 카드나 리스트 항목으로 렌더링한다.
5. browser devtools console에 오류가 없는지 확인한다.

구현 예시

```

async function loadItems() {
  const app = document.querySelector("#app");
  const response = await fetch("data.json");
  const items = await response.json();

  app.innerHTML = items.map((item) => `
    <article class="item">
      <h2>${item.name}</h2>
      <p>Status: ${item.status}</p>
    </article>
  `).join("");
}

loadItems();

```

25-40분 JS fetch/render

- 초점: JSON을 읽고 DOM에 표시한다.

- 학생 산출: 렌더링 코드

흔한 오해

오해	교정
산출물이 있으면 evidence는 나중에 채워도 된다.	evidence는 산출물의 일부다. command, path, status, log, note가 함께 있어야 평가 가능하다.
Week1에서 모든 기술을 깊게 익혀야 한다.	Week1은 컴퓨팅 spine과 운영 증거를 만드는 주차이며, 깊은 hands-on은 각 기술 주차에서 진행한다.
막힌 내용을 숨기는 것이 좋다.	blocker를 증상, 시도한 일, 다음 조치로 기록하는 것이 현업식 진행 관리다.

40-50분 증거 캡처

- 초점: browser 화면과 console 오류를 확인한다.
- 학생 산출: 구현 evidence

산출물

- data rendering이 보이는 browser 화면
- app.js 의 fetch/render 코드
- console 오류 없음 확인 메모

평가 기준

기준	충족
data.json 값이 화면에 표시된다.	
HTML, CSS, JS 책임이 섞이지 않는다.	
console에 blocking error가 없다.	
렌더링 결과를 evidence로 남겼다.	

현업 DevOps insight

운영에서 중요한 것은 "내 컴퓨터에서 봤다"가 아니라 데이터 흐름을 재현 가능한 방식으로 설명하는 것이다. 정적 앱에서도 파일 경로, HTTP 응답, console error는 실제 서비스 운영의 기본 관찰 지점이다.

학술 근거

- Worked example: 먼저 작동하는 작은 예시를 완성한 뒤 변형한다.
- Constructive alignment: 수업 목표, 활동, 평가가 모두 data rendering 증거에 맞춰진다.
- CS2023: client-side programming과 data representation을 함께 다룬다.

다음 주차 연결

Docker preview에서는 container 내부 경로와 browser에서 보이는 HTTP 경로를 구분해야 한다. 오늘의 data.json 경로 확인이 그 준비다.

다음 연결

다음 교시는 사용자 흐름, 필터링 또는 상태 표시, error state를 추가한다.

공식/학술 근거 링크

- RFC 9110: HTTP Semantics, <https://datatracker.ietf.org/doc/html/rfc9110> - request, response, status code evidence의 공식 기준이다.

- MDN HTTP Overview, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> - browser 관찰을 HTTP 흐름으로 설명하는 기준이다.
- Google SRE Book: Introduction, <https://sre.google/sre-book/introduction/> - 상태 확인과 관찰 가능성이 운영 책임에 포함되는 근거다.